# BEM-CUDA Manual

GPU-Accelerated Boundary Element Method for Light Scattering

BEM-CUDA Project

March 2026

## Contents

**1 Overview** **2**

**2 Solvers** **2**
2.1 Dense LU (default) . . . 2
2.2 FMM+GMRES (`-fmm`) . . . 2
2.3 pFFT+GMRES (`-pfft`) . . . 2
2.4 Surface pFFT (`-spfft`) . . . 2

**3 Preconditioners** **2**
3.1 Diagonal (`-prec diag`) . . . 2
3.2 ILU(0) (`-prec ilu0`) . . . 3
3.3 Near-field LU (`-prec nearlu`) . . . 3
3.4 Block-Jacobi (`-prec blockj`) . . . 3

**4 GMRES Variants** **3**
4.1 Standard GMRES(m) . . . 3
4.2 Paired GMRES . . . 3
4.3 GCRO-DR (`-gmres-dr`) . . . 3

**5 Particle Shapes** **4**
5.1 Sphere (`-shape sphere`) . . . 4
5.2 Hex Prism (`-shape hex -ar F`) . . . 4
5.3 OBJ Import (`-obj FILE`) . . . 4

**6 Orientation Averaging** **4**

**7 Output** **4**

**8 Build** **4**

**9 Examples** **4**

**10 References** **5**

# 1 Overview

BEM-CUDA solves the electromagnetic scattering problem for dielectric particles using the Boundary Element Method with PMCHWT formulation. The $2N \times 2N$ system has the structure:

$$\begin{bmatrix} \eta_e L_{\text{ext}} + \eta_i L_{\text{int}} & -(K_{\text{ext}} + K_{\text{int}}) \\ K_{\text{ext}} + K_{\text{int}} & L_{\text{ext}}/\eta_e + L_{\text{int}}/\eta_i \end{bmatrix} \begin{bmatrix} \mathbf{J} \\ \mathbf{M} \end{bmatrix} = \begin{bmatrix} \mathbf{b}_J \\ \mathbf{b}_M \end{bmatrix} \tag{1}$$

where $L$ and $K$ are the single-layer and double-layer BEM operators, $\eta_e = 1$, $\eta_i = 1/|m|$, and $m$ is the complex refractive index.

# 2 Solvers

## 2.1 Dense LU (default)

Direct LU factorization via cuSOLVER. $O(N^3)$ time, $O(N^2)$ memory. Practical for $N \leq 8000$ (ref $\leq 4$).

## 2.2 FMM+GMRES (`-fmm`)

Plane-wave MLFMA with GMRES iterative solver. Matrix-free $O(N \log N)$ matvec. GPU-accelerated P2P, P2M, M2L, L2P kernels.

## 2.3 pFFT+GMRES (`-pfft`)

Precorrected FFT: 3D Cartesian grid with FFT-based far-field. Faster than FMM for smooth geometries due to FFT efficiency.

## 2.4 Surface pFFT (`-spfft`)

Specialized for particles with flat faces (hex prisms). Uses 2D FFT per face instead of 3D FFT. Features:

- FP32 C2C FFT (2× less memory than Z2Z)
- CUDA streams: per-face async kernel execution
- Mixed-radix grids (7-smooth: factors of 2,3,5,7)
- Automatic density-based grid spacing (∼4 points/cell)
- Inter-face P2P kernel for cross-face Green's function

# 3 Preconditioners

Right preconditioning in GMRES: solve $ZM^{-1}(M\mathbf{x}) = \mathbf{b}$.

## 3.1 Diagonal (`-prec diag`)

$z_i = r_i/Z_{ii}$. **Not recommended** for PMCHWT at high $ka$: diagonal elements are nearly pure imaginary, dividing rotates the residual spectrum 90°, *worsening* convergence.

### 3.2 ILU(0) (`-prec ilu0`)

Incomplete LU factorization with zero fill-in on the near-field sparse matrix. Requires ≥30% coverage for effectiveness. Coverage collapses at high refinement due to element-based cell sizing; a geometry-based minimum (`bb_max_dim/5`) prevents this. Build cost scales poorly with coverage ($O(\text{nnz}^2)$).

### 3.3 Near-field LU (`-prec nearlu`)

Full dense LU on the near-field sparse matrix. Best for ref $\leq 3$ with many orientations (9× speedup for 128 orientations). Limited to $N_2 \leq 8000$.

### 3.4 Block-Jacobi (`-prec blockj`)

Spatial cell blocks with dense LU per block.

- Cell size: `bb_max_dim / (2.5 / radius_mult)`, giving ∼8–20 blocks
- **Adaptive splitting**: blocks exceeding 1500 RWG are automatically bisected along the longest axis (recursive, up to 20 rounds)
- **GPU-accelerated apply**: LU factors uploaded to GPU (row-major for coalesced access), CUDA kernel with warp-parallel triangular solve (32 threads/block, `__shfl_down_sync` reduction)
- Auto-fallback to CPU (OpenMP) if GPU memory insufficient
- Build: 5× faster than ILU(0) at ref=4
- Convergence: 21% fewer iterations than ILU(0) at ref=4

**Benchmark** (hex D/L=0.7, $m = 1.3116$, RTX 3080 Ti):

| ref | $ka$ | Precond | Build (s) | Iters | Total (s) |
|---|---|---|---|---|---|
| 3 | 10 | none | – | 451 | 85 |
| 3 | 10 | blockj | 1.4 | 392 | 115 |
| 3 | 10 | ilu0 | 16.4 | 451 | 103 |
| 4 | 5 | none | – | – | 1145 |
| 4 | 5 | blockj | 88 | 333 | 1019 |
| 4 | 5 | ilu0 | 477 | – | 1556 |

## 4 GMRES Variants

### 4.1 Standard GMRES(m)

Restarted GMRES with cuBLAS vector operations and batched Gram–Schmidt orthogonalization (`cublasZgemv`).

### 4.2 Paired GMRES

Two independent GMRES iterations in lockstep for two polarizations, sharing the batched matvec. Halves the number of GPU kernel launches.

### 4.3 GCRO-DR (`-gmres-dr`)

Deflated restarting: recycles a subspace of $k$ Ritz vectors across restarts and across orientations. Benefit: 12–18% at small restart ($m \leq 30$), negligible at $m = 50+$.

# 5 Particle Shapes

## 5.1 Sphere (`-shape sphere`)

Icosphere with configurable refinement level. Refinement $r$: $20 \times 4^r$ triangles, $30 \times 4^r - 60$ RWG basis functions.

## 5.2 Hex Prism (`-shape hex -ar F`)

Hexagonal prism with aspect ratio $H/D = F$. For D/L comparison with ADDA: if $D/L = 0.7$ then `-ar` $= L/D = 1/0.7 \approx 1.4286$. Best used with `-spfft` (surface pFFT exploits flat faces).

## 5.3 OBJ Import (`-obj FILE`)

Load triangulated mesh from Wavefront OBJ file.

# 6 Orientation Averaging

`-orient NA NB NG` specifies a $N_\alpha \times N_\beta \times N_\gamma$ Gauss–Legendre quadrature grid over Euler angles. Typical: `-orient 8 8 1` (64 orientations).

Previous-orientation solution is reused as initial guess for the next, reducing iteration count for neighboring angles.

Far-field is computed in a single GPU-batched call for all orientations and polarizations.

# 7 Output

JSON file with fields:

- `mueller`: $16 \times N_\theta$ array (row-major Mueller matrix elements $M_{11} \ldots M_{44}$)
- `theta`: scattering angles in degrees
- `ka`, `ri_re`, `ri_im`: input parameters
- `time_assembly`, `time_solve`, `time_farfield`, `time_total`: timing (seconds)

# 8 Build

```
make -j$(nproc)
```

For CUDA 12.8+ with gcc 15 (too new), use an older compiler:

```
make -j$(nproc) NVFLAGS="-arch=sm_86 -O3 --use_fast_math \
  -ccbin g++-13 -Xcompiler '-O2 -Wall -std=c++11 -fopenmp' \
  -std=c++11"
```

Set `ARCH = -arch=sm_XX` in `Makefile` for your GPU.

# 9 Examples

```
# Dense LU, sphere, ka=5, single orientation
bin/bem_cuda --ka 5 --ref 3 --ri 1.3116 0 --single

# Surface pFFT, hex D/L=0.7, Block-Jacobi, ka=10
bin/bem_cuda --ka 10 --ref 3 --shape hex --ar 1.4286 \
  --spfft --prec blockj --single

# Orientation-averaged Mueller matrix, 64 orientations
bin/bem_cuda --ka 5 --ref 3 --shape hex --ar 1.4286 \
  --spfft --prec blockj --orient 8 8 1 --out mueller.json
```

## 10 References


1. Rao, Wilton, Glisson, "Electromagnetic scattering by surfaces of arbitrary shape," IEEE TAP, 1982.

2. Chew, Jin, Michielssen, Song, *Fast and Efficient Algorithms in CEM*, 2001.

3. Graglia, "On the numerical integration of the linear shape functions times the 3-D Green's function," IEEE TAP, 1993.

4. Phillips, White, "A precorrected-FFT method for electrostatic analysis," IEEE TCAD, 1997.